

ROS1 零基础自学指导书：自测题与参考答案小册

生成日期：2026-05-24

本小册从《ROS1 零基础自学指导书（最终版）》正文中独立出来，集中收录 10 个主体章节的自测题与参考答案。这样可以减少正文篇幅，也方便学生先完成练习，再对照答案检查理解。

建议使用方式：

1. 学完对应正文章节后，先独立完成本小册中的自测题。
 2. 作答时不要只写结论，应写清原因、命令、观察现象和排障路径。
 3. 完成后再阅读参考答案，检查自己的解释是否覆盖关键事实。
 4. 如果答案中涉及命令，应回到 Ubuntu 20.04 + ROS Noetic 环境中实际验证。
-

第 1 章 Ubuntu 与 Linux 入门：自测题与参考答案

自测题

1. Linux 和 Ubuntu 是什么关系？为什么本书不直接说“安装 Linux”？
2. ROS Noetic 为什么要以 Ubuntu 20.04 作为主线学习环境？
3. `pwd` 的输出为什么会影响后续命令的意义？
4. `>` 和 `>>` 有什么区别？为什么修改 `.bashrc` 时要谨慎？
5. `sudo apt update` 和 `sudo apt install tree` 分别做了什么？
6. 如果 `roscore` 将来提示 `command not found`，应先检查哪三个对象？
7. 为什么不建议在 `~/catkin_ws` 中不加区分地使用 `sudo`？
8. `.bashrc` 修改后，为什么已经打开的终端可能还没有新配置？
9. 原生 Ubuntu、虚拟机、WSL2、Docker 的根本区别是什么？
10. 如果一个教程要求执行 `curl ... | sudo bash`，应先问哪些问题？

参考答案

1. Linux 严格来说是内核，Ubuntu 是基于 Linux 内核并集成系统工具、包管理器、桌面环境和软件仓库的完整发行版。本书说“安装 Ubuntu 20.04”，是因为 ROS Noetic 的官方二进制安装、包名、系统依赖和教程都围绕这个发行版版本展开；只说“安装 Linux”会过于模糊，学生可能装到不匹配的发行版或版本。
2. ROS Noetic 的官方目标平台包含 Ubuntu 20.04 Focal，本书选择它可以让 apt 包、Python 3、C++14、ROS Wiki 教程和社区排障经验保持一致。Ubuntu 22.04/24.04 不是 Noetic 的新手主线目标，在非目标平台安装会引入源码构建、依赖版本和兼容性问题，不适合零基础阶段。
3. `pwd` 显示当前工作目录，而很多命令的意义都依赖当前位置。例如在 `~/catkin_ws` 执行 `catkin_make` 和在 `~/catkin_ws/src/beginner_tutorials` 执行 `catkin_make`，结果完全不同；复制、删除、创建文件也都受当前目录影响。排障时先看 `pwd`，可以避免把文件建到错误位置。
4. `>` 会覆盖目标文件，`>>` 会追加到目标文件末尾。修改 `.bashrc` 时要谨慎，是因为它会影响以后每次打开终端的初始化环境；如果用 `>` 误覆盖 `.bashrc`，可能丢失原有配置。如果要追加 ROS 环境，通常使用 `echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc`，并在追加后用 `tail ~/.bashrc` 检查。

5. `sudo apt update` 更新本机的软件包索引，让 `apt` 知道软件源里有哪些包和版本；它本身通常不安装软件。`sudo apt install tree` 根据当前索引下载并安装 `tree` 包及其依赖。安装 ROS 前必须先确保软件源和索引正确，否则会出现找不到包或安装旧信息的问题。
6. 先查三层：第一，`echo $ROS_DISTRO` 或 `ls /opt/ros/noetic` 判断 ROS 是否安装；第二，`echo $PATH` 和 `which roscore` 判断命令是否在当前 shell 可搜索路径中；第三，`source /opt/ros/noetic/setup.bash` 判断是否只是环境没有加载。如果 `source` 后可用，说明安装可能正常，问题在当前终端环境。
7. 不建议在 `~/catkin_ws` 中不加区分地使用 `sudo`，因为这可能把工作空间中的文件所有者变成 `root`，导致普通用户后续无法编辑、编译或删除生成文件。`catkin` 工作空间通常应由普通用户创建和编译；只有安装系统软件包、改系统目录时才需要 `sudo`。如果权限异常，应先用 `ls -l ~/catkin_ws` 检查所有者。
8. `.bashrc` 是新 Bash 交互 shell 启动时读取的配置文件，已经打开的终端不会自动重新执行它。修改后要么新开终端，要么在当前终端执行 `source ~/.bashrc`。因此“已经写进 `.bashrc`，为什么当前终端还不生效”通常不是配置没写入，而是当前 shell 没重新加载。
9. 原生 Ubuntu 是直接安装在硬盘上的完整系统，性能和硬件访问最好；虚拟机是在现有系统里运行一个完整 Ubuntu，便于快照恢复但图形性能较弱；WSL2 是 Windows 上的 Linux 子系统，适合命令行和轻量实验但 GUI、USB、仿真有额外限制；Docker 是容器化环境，适合复现和隔离，但默认不持久保存工作空间，GUI 和硬件访问需要额外映射。
10. 先问：脚本来自哪里，是否官方或可信？脚本会改哪些文件、安装哪些软件、是否需要 `root` 权限？是否能先打开脚本内容阅读？失败后如何恢复？有没有更透明的手动安装步骤？`curl ... | sudo bash` 把网络下载内容直接交给 `root` 执行，风险比普通命令高，不能仅因教程中出现该命令就直接复制执行。

第 2 章 ROS1 基本概念：自测题与参考答案

自测题

1. 为什么说 ROS 不是传统操作系统？
2. ROS Master 的职责是什么？它不负责什么？
3. Node 和源代码文件有什么区别？
4. Topic 和 Message 的关系是什么？
5. 哪些场景适合 topic？哪些场景适合 service？
6. Action 相比 service 多解决了什么问题？
7. Parameter Server 为什么不适合传输激光雷达数据？
8. 如果 `rostopic echo /scan` 没有输出，应先检查哪两个命令？
9. 为什么发布者和订阅者可以按任意顺序启动？
10. `ROS_MASTER_URI` 对单机和多机通信分别意味着什么？
11. 如果一个节点订阅 `/cmd_vel` 但机器人不动，为什么不能只看 topic 名？
12. 为什么参数变化后节点行为不一定立刻改变？

参考答案

1. ROS 不是传统操作系统，因为它不直接负责进程调度、内存管理、文件系统和硬件驱动这些内核职责。ROS 运行在 Ubuntu/Linux 之上，提供机器人软件常用的通信、构建、包管理、工具和生态。把 ROS 当成操作系统会导致误解，例如以为装了 ROS 就能直接控制电机或读取传感器。
2. ROS Master 负责名称注册和发现：节点向 Master 注册自己发布或订阅的 topic、提供的 service 等信息，其他节点通过 Master 找到连接对象。它不负责转发所有 topic 数据，也不负责执行节点代码、保存传感器数据或保证业务逻辑正确。节点发现彼此后，实际数据连接通常在节点之间建立。

3. 源代码文件是磁盘上的程序文本，例如 `talker.py` 或 `talker.cpp`；Node 是程序运行后加入 ROS 计算图的进程。同一个源码文件可以启动多个节点实例，不同节点名也可能来自同一个可执行文件。排查时应看 `rostopic list` 和 `rostopic info`，不能只看文件是否存在。
4. Topic 是通信通道的名字，Message 是这个通道中传输的数据类型和字段结构。例如 `/chatter` 是 topic，`std_msgs/String` 是消息类型；`/cmd_vel` 是 topic，`geometry_msgs/Twist` 是消息类型。只知道 topic 名不够，还必须确认消息类型一致，否则发布者和订阅者不能正确通信。
5. 持续、异步、高频或多接收者的数据适合 topic，例如激光雷达 `/scan`、里程计 `/odom`、速度命令 `/cmd_vel`。一次请求一次响应的操作适合 service，例如重置仿真、保存地图、查询状态、执行简单计算。判断标准不是“哪个更高级”，而是数据是否持续流动，以及调用方是否需要等待一个明确响应。
6. Action 适合可反馈、可取消、持续时间较长的任务。Service 发出请求后通常等待最终响应，不适合“导航到目标点”这类可能耗时很久、需要中途反馈进度、允许取消的任务。Action 在语义上补充了 goal、feedback、result、cancel 这些机制。
7. Parameter Server 适合存低频配置，例如发布频率、文件路径、阈值、开关参数；激光雷达数据是高频连续数据，应该通过 topic 发布。把 `/scan` 这类数据放入参数服务器会造成更新频率、时间戳、数据同步和性能问题，也不符合 ROS 工具链的观察方式。
8. 先查 `rostopic info /scan`，确认是否有 publisher、subscriber 以及消息类型；再查 `rostopic list` 或 `rostopic info /scan`，确认相关节点和 topic 是否存在。如果 topic 存在但无数据，再用 `rostopic echo /scan`、`rostopic pub` 或节点日志继续定位发布者是否真的在发。
9. 因为发布者和订阅者先向 Master 注册，节点启动顺序不是直接函数调用关系。发布者先启动时可以先注册等待订阅者，订阅者先启动时也可以先注册等待发布者；双方发现彼此后再建立连接。这也是 ROS 系统可以分布式启动的原因。
10. 单机时，`ROS_MASTER_URI` 通常指向本机 Master，例如 `http://localhost:11311`。多机时，所有参与通信的机器必须能访问同一个 Master 地址，并且网络、主机名解析和防火墙要允许节点之间建立连接。只改 `ROS_MASTER_URI` 不一定够，多机还要关注 `ROS_HOSTNAME` 或 `ROS_IP`。
11. 因为 `/cmd_vel` 只是名字，还要确认消息类型是否是 `geometry_msgs/Twist`、是否真的有 publisher、底盘或仿真节点是否订阅、速度值是否非零、控制器是否启用、机器人是否急停或仿真暂停。只看 topic 名会漏掉数据内容、连接关系和执行层问题。
12. 参数服务器只是存储参数，节点是否使用参数取决于代码。很多节点只在启动时读取一次参数，运行中参数变了也不会自动重新读取；有些节点需要重启，有些节点支持动态参数机制。排查时要同时看 `rostopic get` 和节点代码或文档，不能只看参数值是否已经改变。

第 3 章 ROS1 安装方法完整说明：自测题与参考答案

自测题

1. 为什么本书主线不使用 Ubuntu 22.04 安装 Noetic?
2. `desktop-full`、`desktop`、`ros-base` 分别适合什么场景?
3. `sudo apt update` 在 ROS 安装中起什么作用?
4. 为什么添加 ROS 源后要检查 `ros-latest.list`?
5. `source /opt/ros/noetic/setup.bash` 为什么每个终端都要执行?
6. 为什么鱼香 ROS 一键安装不能替代官方安装原理?
7. Docker 容器为什么默认不适合保存学生工作空间?
8. WSL2 中 Gazebo 出问题，为什么不能直接说 ROS 安装失败?

9. rosdep 解决什么问题?
10. 将使用哪些命令判断 ROS Noetic 安装是否成功?
11. 如果 `rosversion -d` 正常但 `turtlesim` 无窗口, 问题可能在哪一层?
12. 为什么源码构建不适合零基础第一次安装?

参考答案

1. 因为 ROS Noetic 的官方目标平台是 Ubuntu 20.04 Focal, 而不是 Ubuntu 22.04。新手使用 Ubuntu 22.04 安装 Noetic, 往往会遇到 apt 包不可用、依赖版本不匹配、需要源码构建等问题。教材主线必须降低无关变量, 让读者先掌握 ROS1 本身。
2. `desktop-full` 包含 ROS 基础、GUI 工具、RViz、rqt、仿真和常用感知/机器人包, 适合本书全流程学习; `desktop` 包含常用桌面工具但内容少一些, 适合空间有限但仍需要 RViz/rqt 的环境; `ros-base` 只有核心通信、构建和命令行基础, 适合服务器、容器或机器人本体, 不适合零基础完整学习图形和仿真。
3. `sudo apt update` 会从 Ubuntu 和 ROS 软件源拉取最新软件包索引。添加 ROS 源或修改镜像后, 如果不执行它, apt 仍不知道新源中有哪些 `ros-noetic-*` 包。安装失败时, apt update 的输出也是判断软件源、网络和 key 是否正常的重要证据。
4. `ros-latest.list` 决定 apt 是否知道 ROS 软件源, 以及源的发行版代号是否匹配当前 Ubuntu。比如 Ubuntu 20.04 应对应 `focal`, 如果文件内容错误或没有写入, `apt install ros-noetic-desktop-full` 就可能找不到包。检查这个文件比重复执行安装命令更有效。
5. `source /opt/ros/noetic/setup.bash` 会把 ROS Noetic 的环境变量加载到当前 shell, 例如 `PATH`、`ROS_PACKAGE_PATH` 等。每个新终端都是新的 shell, 不会自动继承另一个终端里手动 source 的结果。若要新终端自动加载, 需要把 source 行写入 `~/.bashrc`。
6. 鱼香 ROS 一键安装能帮助处理国内网络、换源、rosdep 和自动化安装问题, 但它背后仍然是在操作软件源、apt、rosdep 和环境变量。学生必须理解官方安装原理, 否则一键脚本失败或更换机器时就无法排查。教材可以推荐它作为辅助工具, 但不能把它当成不透明的自动化过程来替代基础知识。
7. Docker 容器默认是临时运行环境, 容器删除后, 容器内部未挂载到宿主机的数据会消失。学生的 `catkin_ws` 如果没有通过 volume 挂载保存, 就可能丢失。Docker 还需要额外处理 GUI、网络、设备映射和权限, 因此更适合作为助教备用环境、批量复现或 CI, 不是第一次学习的最简单路径。
8. WSL2 中 Gazebo 出问题可能来自图形显示、显卡加速、WSLg、网络、权限或资源限制, 不一定是 ROS 安装失败。应先用 `roscore`、`roslaunch`、`rostopic list` 等命令判断 ROS 通信是否正常, 再判断是否是 Gazebo 图形或仿真性能层的问题。
9. rosdep 用来根据 ROS 包声明的依赖, 在当前系统上解析并安装对应系统包。它解决的是“源码或工作空间依赖哪些 Ubuntu 包、Python 包或系统库”的问题。创建或编译较大工作空间时, `rosdep install --from-paths src --ignore-src -r -y` 常用于补齐依赖。
10. 可以用 `rosversion -d` 确认发行版输出 `noetic`, 用 `which roscore` 确认命令可找到, 用 `roscore` 确认 Master 能启动, 用另一个终端 `roslaunch` 确认能连接 Master, 用 `roslaunch turtlesim turtlesim_node` 确认图形工具和示例包可用。这些命令分别验证版本、环境、通信和 GUI 示例。
11. 问题可能在图形层或示例包层, 而不是 ROS 核心安装层。应检查是否安装了 `ros-noetic-turtlesim`、当前是否有桌面显示环境、`DISPLAY` 是否正确、虚拟机/WSL2 图形支持是否正常。`rosversion -d` 只能说明 ROS 环境能识别 Noetic, 不代表 GUI 程序一定能显示。
12. 源码构建会引入仓库分支、依赖解析、编译顺序、系统库版本、构建工具和大量错误日志。零基础学生还没有掌握 apt、source、catkin、依赖声明和排障命令, 直接源码构建会把主要精力耗在环境问题上。源码构建适合作为高级理解和后续适配准备, 不适合作为第一次安装主线。

第 4 章 第一个 ROS 系统：自测题与参考答案

自测题

1. 为什么 turtlesim 适合作为第一个 ROS 系统？
2. /turtle1/cmd_vel 和 /turtle1/pose 分别代表命令还是反馈？
3. rostopic type 和 rosmmsg show 的区别是什么？
4. rostopic pub -1 和 rostopic pub -r 10 的区别是什么？
5. 如果 rqt_graph 看不到 teleop 和 turtlesim 的连接，应先检查哪些命令？
6. 为什么 teleop 节点不需要知道 turtlesim 内部代码？
7. rosnode info 中 Publications 和 Subscriptions 分别代表什么？
8. 如果 rostopic echo /turtle1/cmd_vel 有数据但模拟海龟不动，可能是什么原因？
9. 为什么 rostopic info 有时比 rostopic echo 更适合作为第一步排查？
10. turtlesim 中学到的观察方法如何迁移到移动机器人？

参考答案

1. turtlesim 足够小，启动快、依赖少、现象直观，同时包含节点、topic、message、service、参数和 rqt_graph 等 ROS 核心观察对象。它把真实机器人中的“控制输入”和“状态反馈”简化成模拟海龟运动，适合初学者先理解 ROS 计算图，而不是一开始面对复杂硬件和仿真。
2. /turtle1/cmd_vel 是命令输入，通常由 teleop 或手动 rostopic pub 发布，表示模拟海龟的期望速度；/turtle1/pose 是状态反馈，由 turtlesim 节点发布，描述当前位姿和速度信息。真实移动机器人中，类似关系是 /cmd_vel 作为控制输入，/odom 作为运动反馈。
3. rostopic type /turtle1/cmd_vel 只显示 topic 使用的消息类型，例如 geometry_msgs/Twist；rosmmsg show geometry_msgs/Twist 会展开这个类型的字段结构，例如 linear.x、angular.z。前者解决“这条 topic 是什么类型”，后者解决“这个类型里有哪些字段”。
4. rostopic pub -1 只发布一次消息，适合测试单次命令是否能被接收；rostopic pub -r 10 以 10 Hz 持续发布，适合模拟持续控制输入。速度控制通常需要持续发布，因为很多机器人或仿真系统会在命令停止后逐渐停止或触发安全机制。
5. 先查 rosnode list 确认 teleop 和 turtlesim 节点是否存在，再查 rostopic list 确认 /turtle1/cmd_vel 是否存在，接着用 rostopic info /turtle1/cmd_vel 看 publisher 和 subscriber 是否连接。必要时用 rostopic echo /turtle1/cmd_vel 判断是否真的有速度数据。
6. teleop 节点只需要知道自己向哪个 topic 发布什么消息类型，不需要知道 turtlesim 内部如何更新画面。ROS 的发布订阅模型通过 topic 和 message 解耦节点实现。真实机器人中，键盘控制节点也不需要知道底盘驱动内部如何控制电机，只需要按约定发布 /cmd_vel。
7. Publications 表示该节点发布了哪些 topic，即它向外输出哪些数据；Subscriptions 表示该节点订阅了哪些 topic，即它依赖哪些输入数据。对 turtlesim 来说，它订阅 /turtle1/cmd_vel 并发布 /turtle1/pose，这能直接说明它的输入和输出边界。
8. 可能原因包括：发布的是零速度、消息字段不符合预期、turtlesim 节点没有订阅同一个 topic、节点长时间无响应或窗口未响应、topic 命名空间不一致。排查顺序应先看 rostopic info /turtle1/cmd_vel 是否有 turtlesim 作为 subscriber，再看 echo 出来的速度值是否非零。

9. `rostopic info` 能快速显示 topic 类型、发布者和订阅者，比 `echo` 更适合判断连接关系。`echo` 只能看数据流，如果没有输出，仍无法区分是没有 publisher、没有 subscriber、topic 名错，还是发布频率太低。第一步看 `info`，可以先确定系统结构。
10. 可以把 `/turtle1/cmd_vel` 对应到移动机器人的 `/cmd_vel`，把 `/turtle1/pose` 对应到 `/odom` 或机器人状态，把 `turtlesim` 节点对应为底盘或仿真节点。迁移后的观察顺序仍然是：`roscd` 看节点，`rostopic list` 看接口，`rostopic info` 看连接，`rostopic echo/hz` 看数据，`rqt_graph` 看整体关系。

第 5 章 catkin 工作空间与功能包：自测题与参考答案

自测题

1. 工作空间和功能包是什么关系？
2. 为什么功能包应放在 `~/catkin_ws/src` 下？
3. `catkin_make` 应该在哪个目录执行？为什么？
4. `package.xml` 和 `CMakeLists.txt` 分别解决什么问题？
5. `build/` 和 `devel/` 是谁生成的？为什么不应该把主要源码放进去？
6. 为什么 Python 节点通常不需要像 C++ 那样编译？
7. `source /opt/ros/noetic/setup.bash` 和 `source ~/catkin_ws/devel/setup.bash` 有什么关系？
8. 如果 `rospack find` 找不到包，应按什么顺序排查？
9. 直接依赖和递归依赖有什么区别？
10. 为什么初学阶段不建议在同一个工作空间混用 `catkin_make` 和 `catkin build`？

参考答案

1. 工作空间是一组 ROS 包的工程环境，包含源码空间 `src/`、构建结果 `build/` 和开发环境 `devel/`。功能包是工作空间中的基本代码单元，每个包有自己的 `package.xml` 和 `CMakeLists.txt`。可以把工作空间理解为“项目容器”，把功能包理解为“可复用模块”。
2. `catkin` 约定工作空间的源码空间是 `catkin_ws/src`，`catkin_make` 会扫描这里的包并按依赖构建。包放在其他任意目录中，ROS 工具不一定能找到，`rospack find`、`roscd`、`catkin_make` 都可能失效。遵守目录约定能让教程、工具和排障方法保持一致。
3. `catkin_make` 应该在工作空间根目录执行，也就是包含 `src/` 的 `~/catkin_ws`。它需要从根目录读取源码空间，生成 `build/` 和 `devel/`。如果在包目录或 `src/` 目录执行，`catkin` 可能无法识别工作空间结构，出现“不是 `catkin` 工作空间”或生成结果位置错误。
4. `package.xml` 负责包的元信息和依赖声明，例如包名、维护者、许可证、构建依赖和运行依赖；`CMakeLists.txt` 负责构建规则，例如查找依赖、生成消息、编译 C++ 可执行文件、链接库。前者回答“这个包依赖谁”，后者回答“这个包怎么构建”。
5. `build/` 和 `devel/` 都由 `catkin_make` 生成。`build/` 存放 CMake 缓存和编译中间文件，`devel/` 存放开发环境和生成后的可运行入口。它们是构建产物，不是源码；把主要源码放进去会导致清理构建目录时丢失代码，也不符合 ROS 项目结构。
6. Python 是解释型语言，ROS 中的 Python 节点通常只需要脚本有正确 shebang 和执行权限，运行时由 Python 解释器执行。C++ 需要先编译成机器码并链接 ROS 库，所以必须在 `CMakeLists.txt` 中添加 `add_executable` 和 `target_link_libraries`。这也是 C++ 节点比 Python 多一步构建规则的原因。

7. `/opt/ros/noetic/setup.bash` 加载系统安装的 ROS Noetic 环境, `~/catkin_ws/devel/setup.bash` 在此基础上叠加自建工作空间。通常先 `source` 系统 ROS, 再 `source` 工作空间。后者让当前终端能找到自写包, 同时仍能使用 `/opt/ros/noetic` 中的官方包。
8. 先查包是否真的在 `~/catkin_ws/src` 下; 再回到 `~/catkin_ws` 执行 `catkin_make`; 然后 `source ~/catkin_ws/devel/setup.bash`; 最后检查 `echo $ROS_PACKAGE_PATH` 和 `rospack find` 包名。如果仍找不到, 再检查包名是否拼错、`package.xml` 是否存在、是否把包嵌套在另一个包中。
9. 直接依赖是当前包在 `package.xml` 中明确声明、代码直接使用的包, 例如 `rospy`、`roscpp`、`std_msgs`。递归依赖包含“依赖的依赖”, 数量通常更多。编写包清单时重点声明直接依赖, 不应把所有递归依赖都不加区分地写入清单。
10. `catkin_make` 和 `catkin build` 使用不同的构建工具链和构建目录习惯, 混用会产生缓存、构建空间和配置不一致的问题。零基础阶段排障能力还弱, 统一使用 `catkin_make` 可以让错误路径更少。等理解 `catkin` 工作空间后, 再学习 `catkin_tools` 更合适。

第 6 章 Python 与 C++ 编写 ROS 节点：自测题与参考答案

自测题

1. Python 版本和 C++ 版本的发布订阅通信结构是否相同? 不同点在哪里?
2. 为什么 Python 脚本需要执行权限?
3. 为什么 C++ 节点需要修改 `CMakeLists.txt`?
4. `rospy.spin()` 和循环发布中的 `rate.sleep()` 分别解决什么问题?
5. `ros::spin()` 和 `ros::spinOnce()` 有什么区别?
6. `queue_size` 太小或太大可能有什么影响?
7. Topic 名相同但消息类型不同会发生什么?
8. Service 和 topic 的本质差异是什么?
9. 修改 `.srv` 文件后为什么必须重新 `catkin_make` 并重新 `source`?
10. 如果客户端调用 service 后长时间无响应, 应先检查哪些命令?
11. 为什么 C++ service 节点需要 `add_dependencies`?
12. 如何判断一个节点没有收到数据是发布者问题还是订阅者问题?

参考答案

1. 通信结构相同, 都是节点通过 topic 发布和订阅消息: 发布者向 topic 写数据, 订阅者从 topic 收数据并触发回调。不同点在于 Python 节点由解释器运行, 主要关注脚本权限、导入和运行时错误; C++ 节点需要先编译链接, 主要关注 `CMakeLists.txt`、头文件、类型和链接错误。
2. `roslaunch` 运行 Python 脚本时, 本质上要把脚本当作可执行文件启动。没有执行权限时, Linux 会拒绝执行, 即使脚本内容正确也会报 `Permission denied`。通常用 `chmod +x scripts/*.py` 解决, 同时确保第一行 shebang 是 `#!/usr/bin/env python3`。
3. C++ 源文件不会被 `catkin` 自动猜测并编译。必须在 `CMakeLists.txt` 中写 `add_executable` 告诉 CMake 要生成哪个可执行文件, 再用 `target_link_libraries` 链接 ROS 库。否则 `catkin_make` 可能成功结束, 但不会生成预期的 C++ 节点。
4. `rospy.spin()` 让订阅节点保持运行并处理回调, 否则程序可能创建订阅者后直接退出。`rate.sleep()` 用在循环发布中, 用于按指定频率休眠, 控制发布节奏。一个解决“等待回调”, 一个解决“循环频率”。

5. `ros::spin()` 会进入持续回调处理循环，通常用于纯订阅者或 service server；`ros::spinOnce()` 只处理一次回调，常用于自己写 `while (ros::ok())` 循环的节点。发布者如果还需要处理订阅回调，通常在循环中调用 `spinOnce()` 再 `rate.sleep()`。
 6. `queue_size` 太小，在订阅者或网络短时间跟不上时容易丢弃消息；太大则可能积累旧消息，导致延迟变高。对控制类数据，处理很久以前的速度命令通常没有意义，因此队列不宜过大。选择队列长度要结合消息频率、实时性和是否允许丢旧数据。
 7. ROS 要求同一个 topic 上发布者和订阅者的消息类型一致。如果 topic 名相同但类型不同，连接通常无法正常建立，工具会显示类型不匹配或订阅者收不到数据。排查时要同时用 `rostopic type` 话题名 和代码中的消息类型核对。
 8. Topic 是持续、异步的数据流，适合传感器、状态、速度命令等；Service 是一次请求、一次响应，适合查询、重置、计算等离散操作。Topic 不要求接收方立即回应，service 客户端通常会等待服务端响应。选错通信方式会让系统语义不清，例如用 service 传激光数据就不合适。
 9. `.srv` 文件只是接口定义，必须经过 catkin 的消息生成流程，才能生成 Python 可导入模块和 C++ 头文件。修改后如果不重新 `catkin_make`，代码仍看不到新生成类型；不重新 `source`，当前终端也可能找不到生成结果。因此改 `.srv` 后要编译并重新加载工作空间环境。
 10. 先查 `rosservice list` 看服务是否注册；再查 `rosservice type /服务名` 和 `rossrv show 类型` 看类型是否正确；用 `roscall` 看 server 节点是否运行；必要时查看 server 终端日志。如果服务不存在，客户端通常是在等待服务；如果服务存在但调用失败，要检查请求参数和服务端异常。
 11. `AddTwoInts.h` 这类 C++ 头文件是由 `.srv` 生成的，不是源码目录中一开始就存在的文件。`add_dependencies` 告诉 CMake：编译 service 节点前，必须先完成消息/服务代码生成。否则可能出现偶发的“找不到生成头文件”或并行构建顺序错误。
 12. 先用 `rostopic info 话题名` 看 publisher 和 subscriber 是否都存在。如果没有 publisher，是发布者没启动或 topic 名错；如果没有 subscriber，是订阅者没启动或订阅名错；如果两者都有，再用 `rostopic echo`、`rostopic type`、`rqt_graph` 和节点日志检查数据是否发布、类型是否一致、回调是否处理。
-

第 7 章 ROS 运行管理：自测题与参考答案

自测题

1. 为什么实际 ROS 系统不应该靠手动打开多个终端运行？
2. `roslaunch` 自动启动 `roscore` 是否意味着它就是 ROS Master？
3. `launch` 文件为什么不是顺序脚本？
4. YAML 参数加载到参数服务器后，节点一定会自动使用吗？为什么？
5. 私有参数和全局参数有什么区别？
6. `arg` 和 YAML 分别适合解决什么问题？
7. `remap` 解决什么问题？什么时候只 `remap` 一端会出错？
8. 命名空间适合什么场景？为什么初学阶段不应滥用？
9. `rosbag` 记录的是代码、参数，还是 topic 数据？
10. 为什么初学阶段不建议直接 `rosbag record -a`？
11. 如果 bag 回放后 listener 没输出，应先检查哪些命令？
12. `output="screen"` 对调试有什么帮助？
13. `roswtf` 能解决什么问题？为什么不能完全依赖它？
14. `/use_sim_time=true` 但没有 `/clock` 时，可能出现什么现象？

参考答案

1. 手动打开多个终端容易漏启动节点、漏设参数、顺序不一致、topic 名不统一，也不方便别人复现。实际 ROS 系统需要用 launch、参数文件和记录流程把运行方式固化下来。统一入口能降低人为误差，让排障从“输入过哪些命令”转向“系统配置是否正确”。
 2. 不是。roslaunch 在没有 Master 时可以启动 roscore，但它本身不是 ROS Master。ROS Master 仍然是负责注册和发现的服务；roslaunch 是启动和组织节点、参数、命名空间、remap 的工具。
 3. launch 文件描述要启动哪些进程、加载哪些参数、如何命名和重映射，但不保证业务逻辑按 XML 行顺序完成初始化。某个节点进程启动了，不代表它的 topic、service、TF 或硬件已经就绪。依赖另一个服务时，应在代码中显式等待，例如 wait_for_service，而不是依赖 launch 文件中的先后顺序。
 4. 不一定。YAML 加载到参数服务器，只说明参数已经存储在 ROS 参数系统中；节点是否使用它，取决于节点代码是否读取对应参数名。有些节点只在启动时读取一次，运行中参数改变也不会自动生效。排查时要同时看 rosparam get 和节点代码或文档。
 5. 全局参数使用以 / 开头的完整名字，例如 /use_sim_time；私有参数使用 ~ 语义，通常解析到当前节点命名空间下，例如节点 /talker 的 ~rate 对应 /talker/rate。私有参数适合节点自己的配置，全局参数适合系统级约定，但滥用全局参数容易造成名字冲突。
 6. arg 适合让 launch 文件在启动时可配置，例如 model:=burger、rate:=5.0；YAML 适合保存一组结构化参数，便于版本管理和复用。简单启动选项用 arg 更方便，大量参数或嵌套配置用 YAML 更清晰。二者可以配合使用。
 7. remap 用来在不改代码的情况下改变 topic 或 service 名称解析结果。例如通用节点订阅 scan，运行时可 remap 到 /front_laser/scan。只 remap 一端会出错：发布者发到 /renamed_chatter，订阅者仍听 /chatter，两者就不会连接。此时 rqt_graph 和 rostopic info 能直接看出断开。
 8. 命名空间适合多机器人、多传感器或大型系统分组，例如 /robot1/cmd_vel 和 /robot2/cmd_vel 避免冲突。初学阶段不应滥用，是因为命名空间会改变节点、topic、参数的实际名字，排查时更容易找错路径。先掌握相对名、全局名和私有名，再使用复杂命名空间。
 9. rosbag 记录的是 topic 中传输的消息数据及时间信息，不记录节点源代码，也不完整记录参数服务器状态和仿真世界内部状态。它能回放 /chatter、/scan、/odom 等消息，让订阅者像收到实时数据一样工作，但不能自动复原原来的所有节点逻辑。
 10. rosbag record -a 会录制所有 topic，初学阶段容易生成巨大文件，也会混入很多还不理解的数据，影响分析和存储。更好的做法是先用 rostopic list 和 rqt_graph 看清系统，再只录关键 topic，例如 /cmd_vel、/odom、/scan、/tf。
 11. 先用 rosbag info 文件.bag 确认 bag 中是否包含 listener 订阅的 topic；再用 rostopic list 和 rostopic echo 话题名确认回放时 topic 是否出现；再查 listener 是否启动、订阅的 topic 名和消息类型是否一致。如果 topic 名不一致，可能需要 remap 或改 listener 配置。
 12. output="screen" 会把节点日志直接显示在 roslaunch 终端，初学阶段能立即看到 Python traceback、C++ 错误和 ROS 日志。没有它时，错误可能只写入 ~/.ros/log，学生容易误以为节点“没反应”。调试阶段建议打开 screen 输出，稳定后再按项目需要调整日志方式。
 13. roswtf 能检查环境变量、包路径、ROS 图、依赖和常见配置问题，适合作为第一轮自动诊断。但它只能发现工具规则覆盖到的问题，不能判断业务语义是否正确，例如速度值是否合理、TF 语义是否符合机器人模型、参数是否被节点代码使用。使用 roswtf 后仍要用 CLI 和日志验证具体状态。
 14. 如果 /use_sim_time=true，节点会等待 /clock 提供时间。若没有 Gazebo 或 rosbag play --clock 发布 /clock，依赖 ROS 时间的定时器、时间戳或 TF 查询可能长时间不前进，表现为节点等待、回调不触发或 RViz/TF 时间相关警告。排查时先看 rosparam get /use_sim_time 和 rostopic info /clock。
-

第 8 章 机器人坐标、模型与可视化：自测题与参考答案

自测题

1. 为什么机器人系统不能只用一个全局坐标系？
2. map、odom、base_link 的含义有什么区别？
3. TF 解决什么问题？为什么说它是一棵树？
4. URDF 中 link 和 joint 分别表示什么？
5. joint 的 origin xyz/rpy 决定了什么？
6. robot_state_publisher 为什么需要 URDF 和 /joint_states？
7. joint_state_publisher 在没有真实机器人时有什么作用？
8. RViz 和 Gazebo 的根本区别是什么？
9. 如果 RViz 报 No transform from laser_link to base_link, 应先检查什么？
10. 为什么 RViz 中显示正常不代表 Gazebo 仿真一定正常？

参考答案

1. 因为机器人数据来自不同部件和参考系：激光雷达有自己的坐标系，摄像头有自己的坐标系，底盘有 base_link，地图有 map，里程计有 odom。如果只用一个全局坐标系，传感器安装偏移、机器人运动、地图定位和局部里程计关系都会混在一起，无法正确融合数据。
2. map 是全局地图参考系，通常由建图或定位系统维护，可能因为全局修正发生跳变；odom 是局部连续里程计参考系，短时间连续但会漂移；base_link 是机器人本体坐标系，跟随机器人运动。移动机器人常见链路是 map -> odom -> base_link。
3. TF 解决不同坐标系之间在某个时间点如何转换的问题，例如从 laser_link 转到 base_link，再转到 odom。说它是一棵树，是因为每个 child frame 通常只有一个 parent frame，从根到任意 frame 应有唯一变换路径。若出现多个父节点或断开的 frame，算法和 RViz 都无法可靠转换。
4. link 表示机器人中的刚体部件及其坐标系，例如车体、轮子、激光雷达；joint 表示两个 link 之间的连接关系，包括父子关系、关节类型和相对位姿。URDF 通过 link 和 joint 形成机器人结构树。
5. joint 的 origin xyz/rpy 决定 child link 坐标系相对 parent link 坐标系的位置和姿态。xyz 是平移，rpy 是 roll、pitch、yaw 旋转。轮子位置错误、激光雷达方向不对、模型部件漂移，常常是这里设置不正确。
6. robot_state_publisher 需要 URDF 来知道机器人 link/joint 的结构树，需要 /joint_states 来知道可动关节当前角度或位置。它把结构和关节状态结合起来，发布各 link 之间的 TF。没有 URDF，它不知道机器人有哪些 link；没有关节状态，可动关节位姿无法更新。
7. 在没有真实编码器、控制器或硬件驱动时，joint_state_publisher 可以发布模拟的 /joint_states，让 robot_state_publisher 有输入，从而在 RViz 中显示完整模型。它适合模型调试和教学，不代表真实机器人关节状态。真实机器人中 /joint_states 应来自驱动或控制器。
8. RViz 是可视化工具，用来显示 ROS 中已有的数据，例如 TF、RobotModel、LaserScan、Map；Gazebo 是仿真器，用来模拟物理世界、机器人运动、碰撞、重力和传感器。RViz 能“看见”机器人不代表机器人有物理行为，Gazebo 才负责产生仿真运动和传感器数据。
9. 先检查 frame 名是否写对，例如实际可能叫 laser、base_scan 而不是 laser_link；再用 rostopic echo /tf_static 和 rosrunc tf view_frames 看是否存在 base_link 到 laser_link 的路径；还要检查 URDF 中 parent/child 是否连上，以及 robot_state_publisher 是否运行。

10. RViz 只显示模型和 ROS 数据，不验证 collision、inertial、关节控制器、Gazebo 插件、轮地摩擦和物理稳定性。一个只有 visual 的 URDF 可以在 RViz 中很好看，但在 Gazebo 中可能没有质量、碰撞体或控制插件，无法真实运动。因此 RViz 正常只是模型和 TF 可视化正常，不等于仿真物理正确。

第 9 章 仿真与移动机器人入门：自测题与参考答案

自测题

1. RViz 和 Gazebo 的根本区别是什么？
2. /cmd_vel、/odom、/scan 分别属于控制输入还是状态/传感器输出？
3. 为什么移动机器人必须维护 TF？
4. 如果 /cmd_vel 有数据但机器人不动，应按什么顺序排查？
5. rostopic hz /scan 能帮助判断什么？
6. SLAM、定位、导航三者的关系是什么？
7. 为什么本书不在本章展开复杂导航算法？
8. 为什么 TurtleBot3 适合作为教学案例？
9. bag 能记录移动机器人实验的哪些内容？不能记录哪些内容？
10. Gazebo 运行缓慢时，为什么不能立刻判断 ROS 通信出错？

参考答案

1. RViz 显示 ROS 数据，例如模型、TF、LaserScan、Odometry 和 Map；Gazebo 模拟物理世界、机器人运动、碰撞、传感器和环境交互。简单说，RViz 是“看数据”，Gazebo 是“产生仿真数据并模拟物理”。两者经常一起用，但职责不同。
2. /cmd_vel 是控制输入，通常由键盘、导航或工具节点发布，消息类型常为 geometry_msgs/Twist。/odom 是状态输出，通常由底盘或仿真发布，表示局部里程计位姿和速度。/scan 是传感器输出，通常由激光雷达或仿真插件发布，表示二维激光距离数组。
3. 移动机器人中的激光、车体、轮子、里程计和地图都处在不同坐标系中。TF 让系统知道 /scan 属于哪个激光 frame、机器人本体在 odom 中的位置、地图和里程计之间如何关联。没有 TF，RViz 可能无法显示数据，SLAM 和导航也无法正确融合传感器与位姿。
4. 先用 rostopic info /cmd_vel 看是否有 Gazebo 或底盘控制节点订阅；再看 rostopic echo /cmd_vel 中速度值是否非零；然后检查 Gazebo 是否暂停、模型控制插件是否加载、终端是否报错；最后检查 topic 名是否被命名空间或 remap 改变。不能只因为 /cmd_vel 有数据就认定控制链路完整。
5. rostopic hz /scan 能判断激光雷达数据是否持续发布，以及发布频率是否大致稳定。如果频率为 0 或长时间无输出，说明传感器插件、仿真状态或 topic 连接可能有问题；如果频率很低，可能是仿真性能不足或系统负载过高。
6. SLAM 同时估计机器人位姿并建立地图，通常输出 /map 和 map -> odom 相关 TF；定位是在已有地图中估计当前位姿，例如 AMCL；导航根据地图、定位、目标点和障碍物规划路径，并最终输出 /cmd_vel。它们关系是：建图得到地图，定位使用地图估计位置，导航使用位置和地图生成运动命令。
7. 本书目标是 ROS1 入门，不是算法教材。SLAM、定位和导航内部涉及概率、优化、栅格地图、代价地图和路径规划等复杂内容，如果本章展开会偏离主线。这里重点是理解这些模块在 ROS 系统中的输入、输出和连接方式，为后续深入算法打基础。
8. TurtleBot3 有成熟的开源模型、仿真世界、键盘控制、SLAM 和导航示例，topic 和 TF 结构也适合教学。学生可以先在仿真中观察 /cmd_vel、/odom、/scan、/tf，再逐步理解真实移动机器人系统。它的资料和社区问题较多，也便于排障。

9. bag 能记录 topic 消息，例如 /cmd_vel、/odom、/scan、/tf、/tf_static 和 /rosout，用于回放传感器与状态数据。它不能完整记录 Gazebo 世界内部状态、节点源代码、所有参数变化、键盘操作意图或外部文件。要复现实验，还需要 README、launch、参数和依赖说明。
10. Gazebo 卡顿可能是图形性能、CPU/GPU 资源、虚拟机加速、仿真实时率或显示环境问题，不等于 ROS 通信出错。应先用 rostopic list、rostopic hz /scan、rostopic echo /odom 判断数据是否仍在发布。如果 topic 正常但画面慢，问题更可能在仿真性能层。

第 10 章 综合项目：自测题与参考答案

自测题

1. 为什么综合项目必须有一个统一启动入口？
2. 为什么建议把 description、bringup、tools 分成不同包？
3. /cmd_vel 有数据是否一定说明机器人会动？为什么？
4. 一个 bag 文件能复现实验的哪些部分？不能复现哪些部分？
5. 如果 RViz 报 Fixed Frame 错误，应如何排查？
6. 为什么项目 README 要写节点、topic、参数表？
7. 什么样的项目算“只能在作者电脑上运行”？
8. 如果要把本项目交给同学运行，最少要提供哪些信息？
9. 为什么综合项目不要求完整自动导航？
10. 如何判断一个项目是“功能堆叠”还是“系统结构清楚”？
11. 为什么不应把 build/、devel/ 和大型 bag 文件直接提交进仓库？

参考答案

1. 统一启动入口能把节点、参数、命名空间、仿真和可视化配置固化下来，避免靠作者记忆手动打开多个终端。它让别人可以用同一条命令复现主体系统，也让排障有明确起点。没有统一入口的项目，较难判断失败是配置问题、命令遗漏还是代码问题。
2. description 放模型和可视化配置，bringup 放系统启动和参数，tools 放测试脚本和辅助节点。这样分包后，每个包职责清楚，修改 URDF 不会影响启动逻辑，修改工具脚本也不会污染模型描述。成熟 ROS 项目常用类似分层，便于协作和复用。
3. 不一定。/cmd_vel 有数据只说明有人发布了速度命令，还要确认底盘或 Gazebo 控制插件是否订阅、速度值是否非零、机器人是否急停、仿真是否暂停、topic 名是否一致、控制器是否正常。必须用 rostopic info /cmd_vel 看订阅者，用 Gazebo/RViz 和 /odom 看运动反馈。
4. bag 能复现 topic 数据流，例如速度命令、里程计、激光、TF 和日志，使订阅者能在没有原始传感器时接收相同消息。它不能复现源代码、完整依赖环境、Gazebo 内部状态、所有参数文件和启动顺序。因此 bag 必须配合 launch、README、参数和依赖说明，才算可复现实验材料。
5. 先看 RViz 中 Fixed Frame 写的 frame 名是否真实存在；再用 rosrn tf view_frames 或 rostopic echo /tf、/tf_static 检查 TF 树；然后检查 RobotModel、LaserScan 或 Odometry 的 frame_id 是否能连接到 Fixed Frame。若是 TurtleBot3，常见选择是 odom 或 base_link，建图导航时才常用 map。
6. README 中的节点、topic、参数表让项目结构可检查、可交流、可复现。别人不应该通过猜 launch 文件来理解系统，而应从 README 知道每个节点做什么、发布和订阅什么、参数在哪里、如何录 bag、常见错误怎么排查。写文档也是检验自己是否准确理解系统的方式。

7. 只能在作者电脑上运行的项目通常依赖隐含环境：没有写依赖包、没有统一 launch、参数散落在终端历史里、路径写死到个人目录、bag 和 RViz 配置缺失、README 只写一句“运行即可”。换一台机器后，如果别人无法按文档安装、启动、观察和排障，就说明可复现性不合格。
8. 至少提供：Ubuntu 和 ROS 版本、依赖包安装方式、工作空间结构、编译命令、启动命令、关键节点/topic/参数说明、RViz/Gazebo 观察方法、bag 录制与回放命令、常见错误和检查命令。如果使用 TurtleBot3，还要说明模型变量、仿真包来源和 noetic 分支要求。
9. 完整自动驾驶涉及地图、定位、代价地图、全局规划、局部规划、恢复行为和大量参数调优，已经超出 ROS1 入门上册目标。综合项目的重点是证明学生能组织系统、观察数据、控制机器人、记录实验和解释结构。先把工程闭环做稳，再深入导航算法更合理。
10. 功能堆叠的项目通常 launch 很多东西但解释不清节点关系、topic 方向、参数来源和错误定位；系统结构清楚的项目能列出数据流，能说明每个包职责，能用 CLI 验证关键接口，能录制和回放数据，能让别人按 README 复现。判断标准不是功能数量，而是结构是否可解释、可观察、可维护。
11. build/、devel/ 是构建产物，可由源码重新生成，提交后会造成仓库噪声和跨机器路径问题；大型 bag 文件会迅速增大仓库体积，也不一定是每个读者复现实验的必要文件。更好的做法是提交源码、launch、参数、RViz 配置、README 和 rosbag info 输出；确有必要的数据集应单独归档并在 README 中说明获取方式。